

SIMATS ENGINEERING

COMMON ASSESSMENT

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1109

COURSE OUTCOME COVERED

CO1: *Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.*

CO2: *Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.*

CO3: *Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.*

Assignment Weightage: 100%

QUESTIONS: 1

TotalMarks:100

Annexure A

Assignment

Code of Conduct:

I _____ Akash A _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Akash

Annexure A

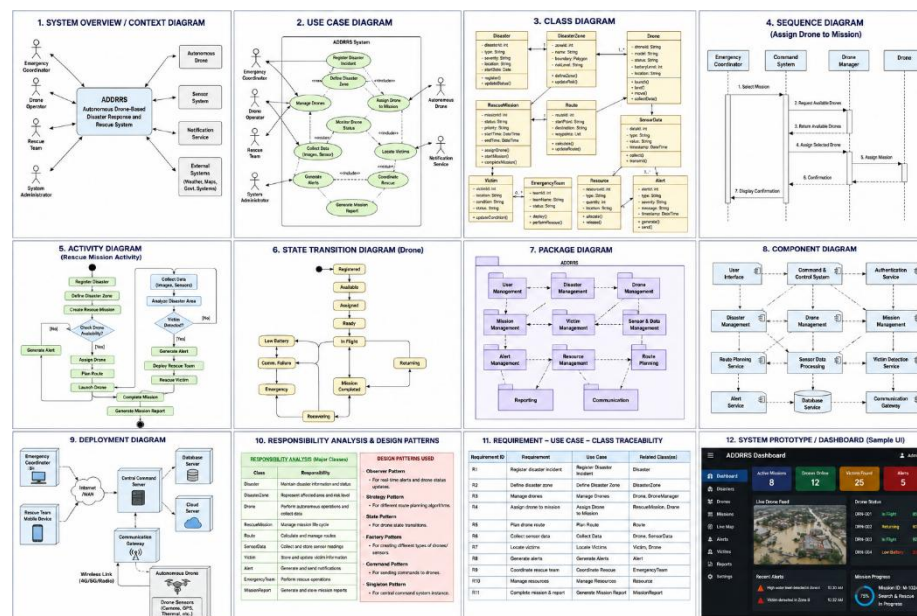
Assignment

1.Object-Oriented Analysis and UML-Based Design of an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)

1. Problem Analysis and Requirement Summary

Problem Statement:

The Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) is designed to assist emergency teams during disasters such as floods, earthquakes, fires, and landslides. The system uses autonomous drones equipped with cameras and sensors to collect real-time information about affected areas, locate victims, monitor environmental conditions, and transmit data to a central command system.



Functional Requirements:

- Register and manage disaster incidents.
- Define and manage disaster zones.
- Register and monitor drones.
- Assign drones to rescue missions.
- Plan and update drone routes.
- Collect images, videos, GPS locations, and sensor readings.
- Locate and record victims.
- Monitor drone status in real time.
- Generate emergency alerts.
- Coordinate emergency teams and resources.
- Handle drone failures and communication loss.

Non-Functional Requirements:

- Real-time communication.
- High reliability and availability.
- Secure access for authorized personnel.
- Scalability for multiple disasters and drones.
- Fast response time.
- Maintainability and flexibility.
- Fault tolerance and failure handling.

2. Actor Identification

Actor	Responsibility
Emergency Coordinator	Registers incidents and manages rescue operations
Drone Operator	Monitors and controls drones when required
Rescue Team	Performs physical rescue operations
System Administrator	Manages users, drones, and system configuration
Autonomous Drone	Collects data and performs assigned missions
Central Command System	Processes and manages disaster information
Victim	Person identified as requiring rescue
Sensor System	Provides environmental readings
Notification Service	Sends emergency alerts and notifications

3. Use Case Descriptions

Major use cases:

1. Register Disaster Incident
2. Define Disaster Zone
3. Register Drone
4. Assign Drone to Mission
5. Plan Drone Route
6. Launch Drone
7. Monitor Drone

8. Collect Sensor Data
9. Capture Images/Videos
10. Detect/Locate Victim
11. Generate Alert
12. Coordinate Rescue Team
13. Manage Resources
14. Handle Drone Failure
15. Complete Rescue Mission
16. Generate Mission Report

Example Use Case

Use Case: Assign Drone to Rescue Mission

Primary Actor: Emergency Coordinator

Precondition: Disaster incident and drone are registered.

Main Flow:

1. Coordinator selects a rescue mission.
2. System displays available drones.
3. Coordinator selects a suitable drone.

4. Use Case Diagram

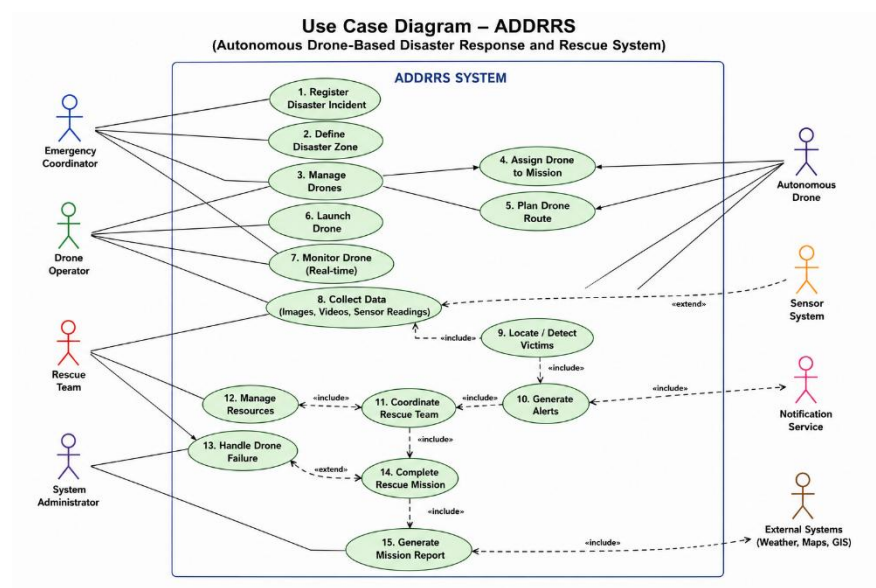
The Use Case Diagram should contain the ADDRRS system boundary.

Major use cases inside the boundary:

- Manage Disaster
- Manage Disaster Zone
- Manage Drones
- Assign Mission
- Plan Route

- Launch Drone
- Monitor Drone
- Collect Data
- Locate Victim
- Generate Alerts
- Coordinate Rescue
- Manage Resources
- Handle Failure
- Generate Reports

External actors should be placed outside the system boundary.



5. Object and Class Identification

Major Objects

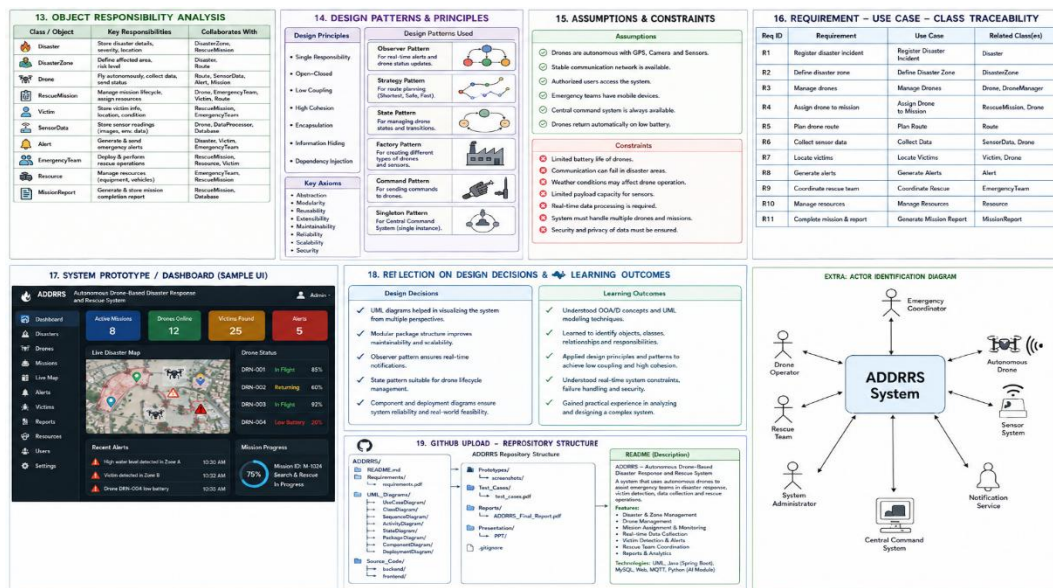
- Disaster
- DisasterZone
- Drone

- DroneOperator
- RescueMission
- Victim
- EmergencyTeam
- SensorData
- Alert
- Resource
- Route
- MissionReport
- EmergencyCoordinator

Major Classes

Class	Important Attributes	Important Methods
Drone	droneId, status, location, batteryLevel	launch(), land(), move(), collectData()
Disaster	disasterId, type, severity, location	register(), updateStatus()
DisasterZone	zoneId, boundary, riskLevel	defineZone(), updateRisk()
RescueMission	missionId, status, priority	assignDrone(), startMission(), completeMission()
Victim	victimId, location, condition	updateCondition(), requestRescue()
EmergencyTeam	teamId, teamName, status	deploy(), performRescue()
SensorData	dataId, type, value, timestamp	collect(), transmit()

Class	Important Attributes	Important Methods
Alert	alertId, type, severity, message	generate(), send()
Route	routeId, startPoint, destination	calculate(), updateRoute()
Resource	resourceId, type, quantity	allocate(), release()
MissionReport	reportId, summary, timestamp	generate(), store()



6. Class Diagram

The Class Diagram should show relationships such as:

- Disaster → DisasterZone
- Disaster → RescueMission
- RescueMission → Drone

- RescueMission → EmergencyTeam
- Drone → SensorData
- Drone → Route
- Drone → Alert
- RescueMission → Victim
- EmergencyTeam → Resource
- RescueMission → MissionReport

Important relationships:

- Association
- Aggregation
- Composition
- Generalization
- Dependency

Example:

Disaster

```
|
| 1..*
|
```

DisasterZone

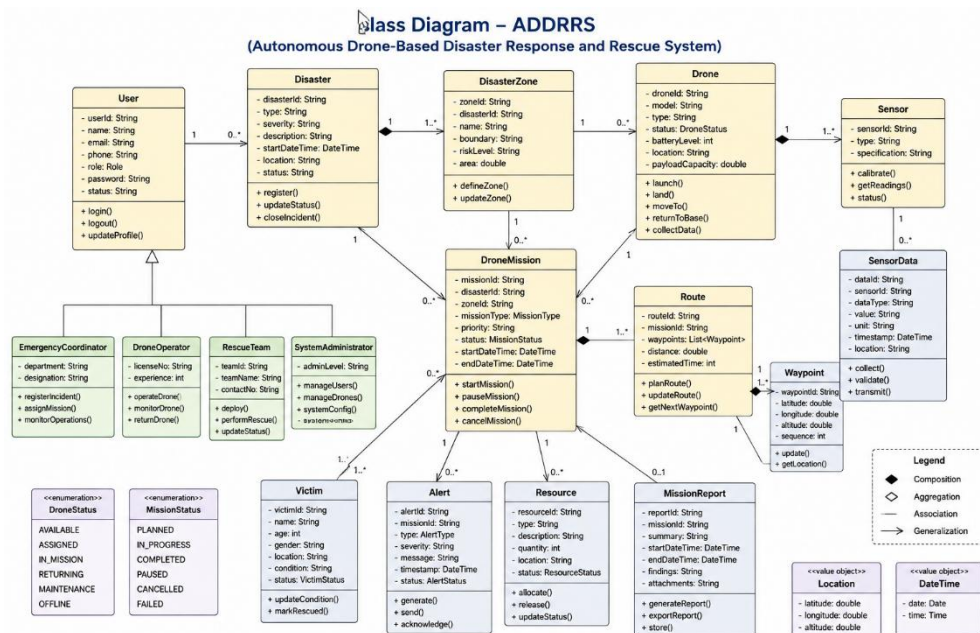
RescueMission

```
|----- Drone
|----- EmergencyTeam
|----- Victim
|----- Route
|----- MissionReport
```

Drone

|----- SensorData

|----- Alert



7. Sequence / Interaction Diagrams

You can create 2–3 sequence diagrams.

Sequence Diagram 1: Assign Drone to Rescue Mission

Emergency Coordinator

|
| Select Mission

v

Command System

|
| Request Available Drones

v

Drone Manager

|

| Return Available Drone

v

Command System

|

| Assign Drone

v

Drone

|

| Assignment Confirmation

v

Command System

|

| Display Confirmation

v

Emergency Coordinator

Sequence Diagram 2: Autonomous Victim Detection

Drone

|

| Capture Image

v

Camera/Sensor

|

| Image Data

v

Command System

|

| Analyze Data

v

Victim Detection Module

|

| Victim Detected

v

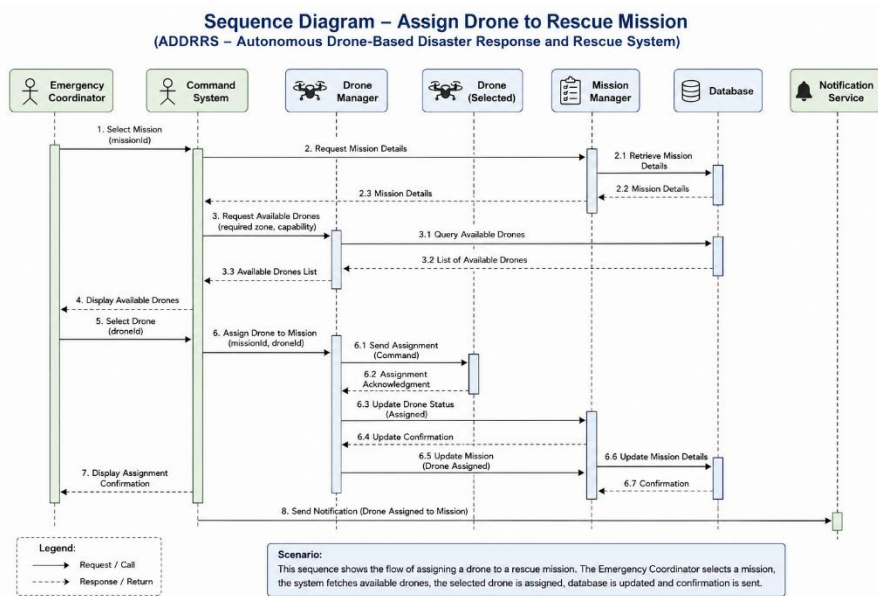
Alert Manager

|

| Emergency Alert

v

Rescue Team



8. Activity Diagram

Rescue Mission Activity

Start



Register Disaster



Define Disaster Zone



Create Rescue Mission



Check Drone Availability



Assign Drone



Plan Route



Launch Drone



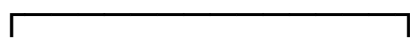
Collect Data



Analyze Disaster Area



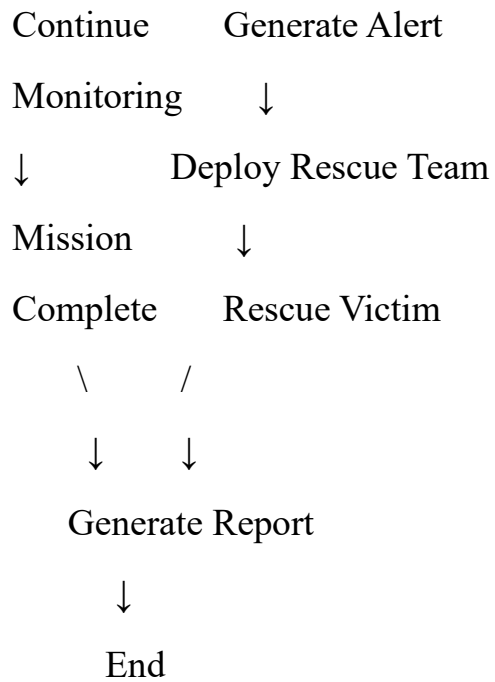
Victim Detected?



No

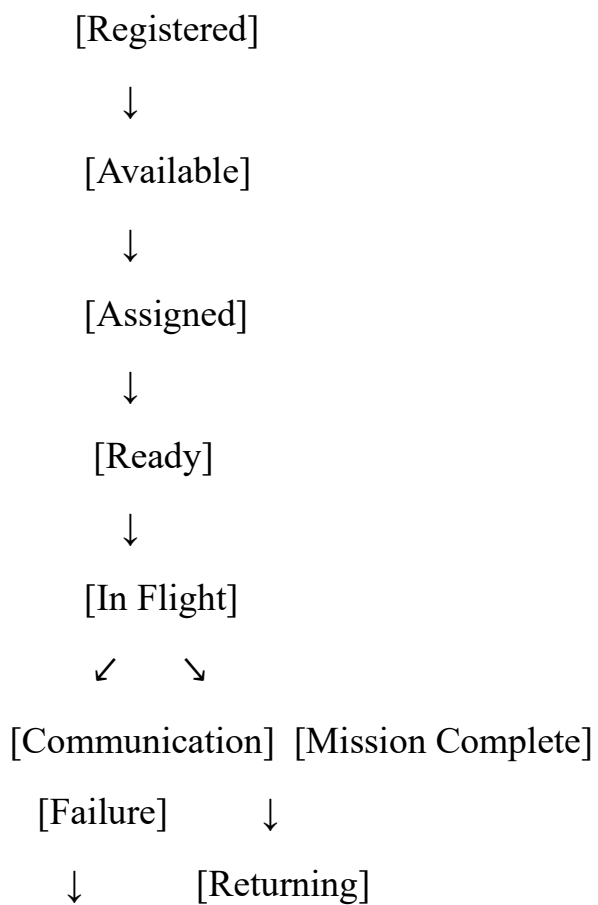
Yes

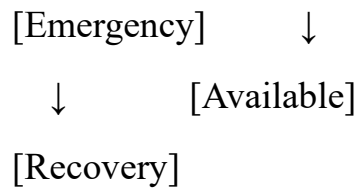




9. State Transition Diagram

The **Drone** is a suitable object for the State Transition Diagram.





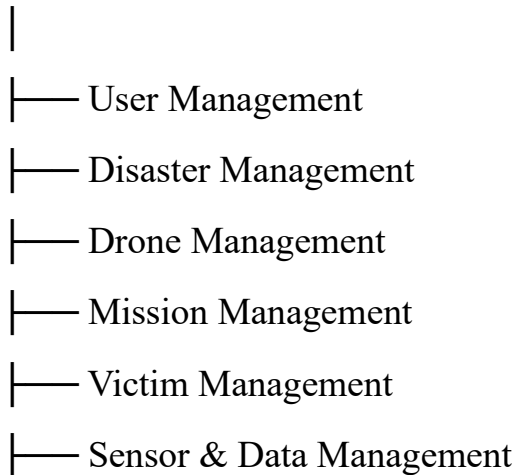
Possible drone states:

- Registered
- Available
- Assigned
- Ready
- In Flight
- Returning
- Mission Completed
- Low Battery
- Communication Failure
- Emergency
- Maintenance

10. Package Diagram

Suggested packages:

ADDRRS



- |— Alert Management
- |— Resource Management
- |— Route Planning
- |— Reporting
- └— Communication

This improves **modularity and maintainability**.

11. Component Diagram

Major components:

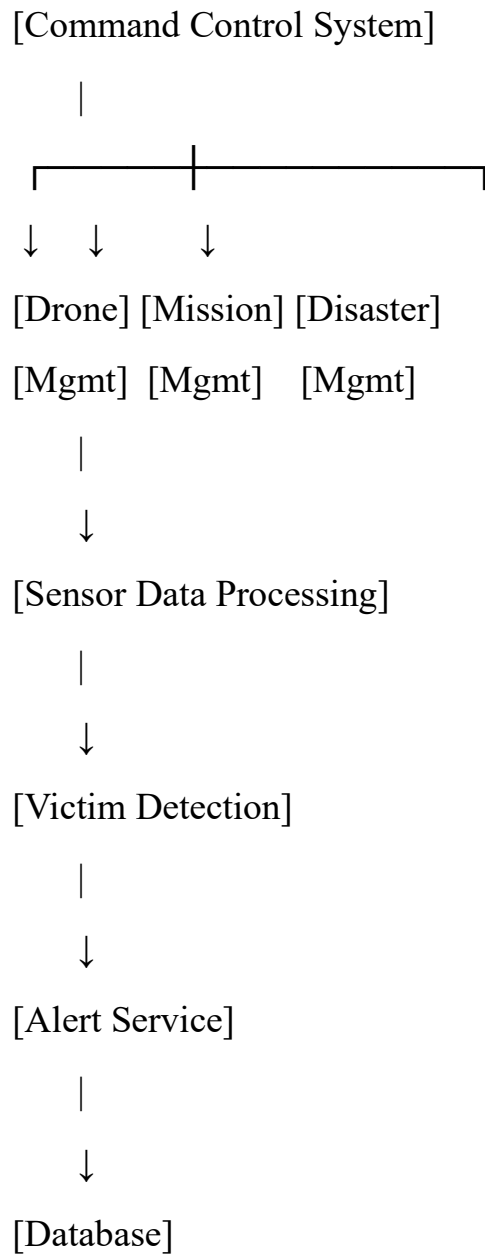
- User Interface
- Command Control System
- Authentication Service
- Disaster Management
- Drone Management
- Mission Management
- Route Planning Service
- Sensor Data Processing
- Victim Detection
- Alert Service
- Database
- Communication Gateway
- Drone Controller

Example:

[User Interface]

|

v



12. Deployment Diagram

The Deployment Diagram can contain:

Devices/Nodes

- Emergency Coordinator Computer
- Rescue Team Mobile Device
- Central Command Server
- Database Server

- Communication Gateway
- Autonomous Drone
- Drone Sensors
- Cloud Server

Example:

[Emergency Coordinator PC]

|

| Internet

↓

[Central Command Server]

|

└───┘

↓

↓

[Database] [Cloud Server]

|

↓

[Communication Gateway]

|

| Wireless Network

↓

[Autonomous Drone]

|

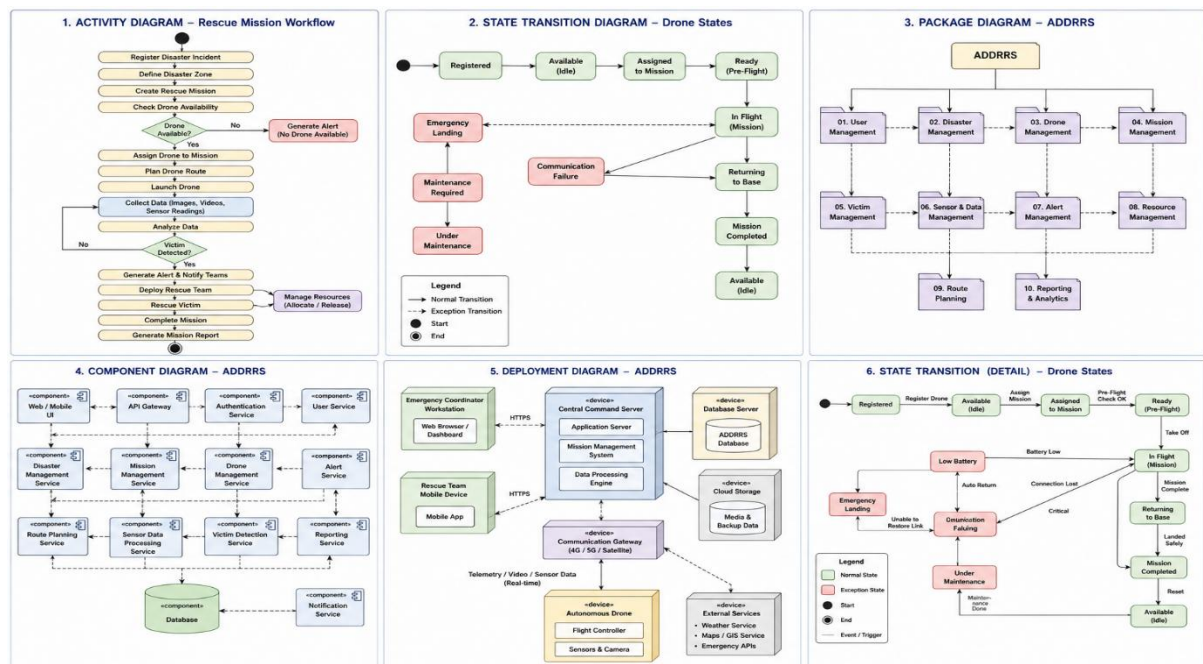
└───┘

↓

↓

↓

Camera GPS Sensors



13. Object Responsibility Analysis

Object/Class	Responsibility
Drone	Execute autonomous flight and collect information
RescueMission	Manage mission lifecycle
Disaster	Store disaster information
DisasterZone	Represent affected geographical area
Route	Calculate and manage drone routes
Victim	Maintain victim information
SensorData	Store environmental readings
Alert	Manage emergency notifications
EmergencyTeam	Perform rescue activities
Resource	Manage rescue resources

Object/Class	Responsibility
MissionReport	Generate mission completion information
DroneManager	Manage drone availability and assignments

Responsibility Assignment

The design follows high cohesion by giving each class a focused responsibility and low coupling by minimizing unnecessary dependencies between classes.

14. Design Axioms, Principles and Design Patterns

Design Principles

1. High Cohesion

Each class performs closely related operations.

2. Low Coupling

Classes depend on interfaces rather than unnecessary concrete implementations.

3. Single Responsibility Principle

Each class has one major responsibility.

4. Open/Closed Principle

New drone or sensor types can be added without major changes to existing code.

5. Encapsulation

Internal data such as drone status and battery level is protected through appropriate methods.

Suitable Design Patterns

Pattern	Application
Observer Pattern	Real-time drone status and alert notifications
Strategy Pattern	Different route-planning algorithms

Pattern	Application
Factory Pattern	Creating different drone/sensor objects
State Pattern	Managing drone states
Command Pattern	Sending commands to autonomous drones
Singleton Pattern	Central command/control manager, if a single shared instance is required

15. Assumptions and Constraints

Assumptions

- Only authorized personnel can access the system.
- Drones have GPS capability.
- Drones have cameras and environmental sensors.
- A communication network is available in most operational areas.
- Emergency teams have suitable communication devices.
- The central system maintains a database.
- Drones can operate autonomously after mission assignment.

Constraints

- Limited drone battery capacity.
- Possible communication failures.
- Poor weather conditions may affect drone operation.
- GPS signals may become unavailable.
- Network bandwidth may be limited.
- System must handle multiple drones simultaneously.
- Security must prevent unauthorized access.
- Real-time data processing is required.

16. Requirement-to-Use Case/Class Traceability

Requirement	Use Case	Related Classes
Register disaster	Register Disaster	Disaster
Define affected area	Define Disaster Zone	DisasterZone
Manage drones	Manage Drone	Drone, DroneManager
Assign drone	Assign Drone	Drone, RescueMission
Plan route	Plan Route	Route
Collect sensor data	Collect Data	Drone, SensorData
Locate victims	Locate Victim	Victim, Drone
Generate emergency notification	Generate Alert	Alert
Coordinate rescue	Coordinate Rescue	EmergencyTeam, RescueMission
Manage resources	Manage Resources	Resource
Complete mission	Complete Mission	RescueMission, MissionReport

17. Implementation / Prototype and Result:

A simple prototype can demonstrate:

- Login page
- Disaster registration
- Drone registration
- Drone status dashboard
- Mission assignment
- Disaster-zone display
- Victim information
- Sensor-data monitoring
- Emergency alerts
- Mission report generation

Expected Result

The prototype should demonstrate that emergency personnel can register disasters, assign drones, monitor autonomous operations, receive collected information, identify victims, generate alerts, coordinate rescue teams, and complete rescue missions.

18. Reflection on Design Decisions and Learning Outcomes

The development of ADDRRS demonstrates the application of Object-Oriented Analysis and Design principles to a real-world emergency-response problem. UML diagrams helped identify system actors, objects, classes, relationships, workflows, interactions, states, components, and deployment nodes.

The project provided practical understanding of:

- Object identification and classification.
- Use Case Modelling.

- UML diagram development.
- Class and relationship identification.
- Responsibility assignment.
- Object-oriented design principles.
- Design patterns.

19. GitHub Upload

<https://github.com/akkurthiakash/CSA1109>